

11 Software Testing - White Box

Program Graph / Control Flow Graph

程序图 / 控制流图

- DD-Paths
- Test Coverage Metrics
- Basis Path Testing

Guidelines and Observations

White Box Testing

- Also known as glass box, structural, clear box and open box testing.
也称为玻璃箱、结构、透明盒和开箱测试。
- A software testing technique whereby explicit knowledge of the internal workings of the item being tested are used to select the test data.
白盒测试：一种软件测试技术，利用对被测试项目内部工作原理的明确知识来选择测试数据。
- Unlike black box testing that is using the program specification to examine outputs, **white box testing is based on specific knowledge of the source code to define the test cases** and to examine outputs.

The nature of white box testing allows for:

- **Rigorous Definitions:** In white box testing, testers have complete knowledge of the software's internal logic, including the code, algorithms, and data structures. This allows for a high level of specificity in defining what is to be tested and how.
严格定义：在白盒测试中，测试人员全面了解软件的内部逻辑，包括代码、算法和数据结构。这使得在确定测试内容及方法时能够达到高度的精确性。
- **Mathematical Analysis:** The inner workings of software can often be represented mathematically, using concepts like graphs, trees, and matrices. This allows testers to apply mathematical analysis methods for better understanding and assessment of the code.
数学分析：软件的内在运作常可通过图形、树形结构与矩阵等数学概念加以表征。这使得测试人员能够运用数学分析方法，更深入地理解与评估代码。
- **Useful Measurement:** Since white box testing involves testing the internal structures of the application, it provides opportunities to gather detailed metrics about the code. These can include metrics related to code coverage, path coverage, and complexity. This data can be invaluable in identifying problem areas, optimizing performance, and improving overall code quality.
实用度量：由于白盒测试涉及对应用内部结构的测试，因此能够收集关于代码的详细度量指标。这些指标包括代码覆盖率、路径覆盖率和复杂度相关数据。此类数据对于识别问题区域、优化性能以及提升整体代码质量具有不可估量的价值。

Program Graphs

- Given a program written in an imperative programming language, its program graph is a directed graph in which nodes are statement fragments, and edges represent flow of control.

给定一个用命令式编程语言编写的程序，其程序图是一个有向图，其中节点表示语句片段，边表示控制流。

- A.k.a. Control Flow Graph
- If i and j are nodes in the program graph, an edge exists from node i to node j **if and only if** the statement fragment corresponding to node j can be executed immediately after the statement fragment corresponding to node i .

如果 i 和 j 是程序图中的节点，那么当且仅当节点 j 对应的语句可以在节点 i 对应的语句片段之后立即执行时，从节点 i 到节点 j 存在一条边。

- In short:
 - In the context of white box testing, a program graph is used **to visualize all possible execution paths** in a piece of code. Each node in the graph represents program statement or statement fragment, and each directed edge represents the transition from one state to another.

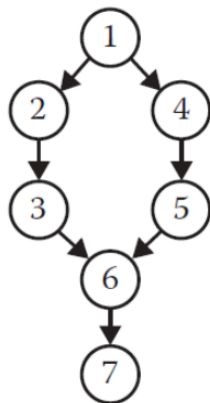
在白盒测试的情境下，程序图用于可视化代码中所有可能的执行路径。图中的每个节点代表一条程序语句或语句片段，每条有向边则表示从一种状态到另一种状态的转换。

Basic Structure of Program Graph

- 4 basic structures

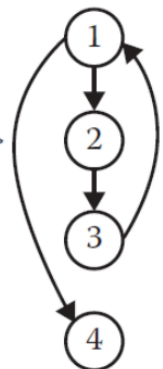
If-Then-Else

```
1 If <condition>
2   Then
3     <then statements>
4   Else
5     <else statements>
6 End If
7 <next statement>
```



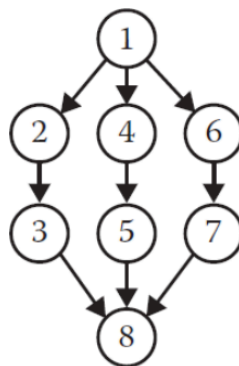
Pretest loop

```
1 While <condition>
2   <repeated body>
3 End While
4 <next statement>
```



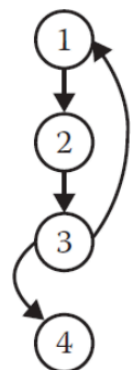
Case/Switch

```
1 Case n of 3
2   n=1:
3     <case 1 statements>
4   n=2:
5     <case 2 statements>
6   n=3:
7     <case 3 statements>
8 End Case
```



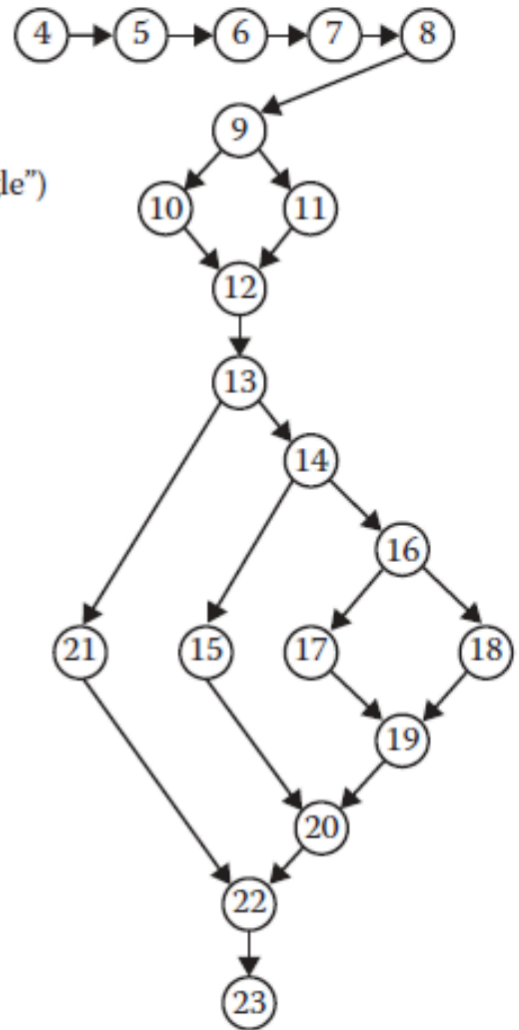
Posttest loop

```
1 Do
2   <repeated body>
3 Until <condition>
4 <next statement>
```



Example

```
1 Program triangle2
2 Dim a,b,c As Integer
3 Dim IsATrinagle As Boolean
4 Output("Enter 3 integers which are sides of a triangle")
5 Input(a,b,c)
6 Output("Side A is", a)
7 Output("Side B is", b)
8 Output("Side C is", c)
9 If (a < b + c) AND (b < a + c) AND (c < a + b)
10 Then IsATriangle = True
11 Else IsATriangle = False
12 EndIf
13 If IsATriangle
14 Then If (a = b) AND (b = c)
15       Then Output ("Equilateral")
16       Else If (a≠b) AND (a≠c) AND (b≠c)
17             Then Output ("Scalene")
18             Else Output ("Isosceles")
19       EndIf
20 EndIf
21 Else Output("Nota a Triangle")
22 EndIf
23 End triangle2
```

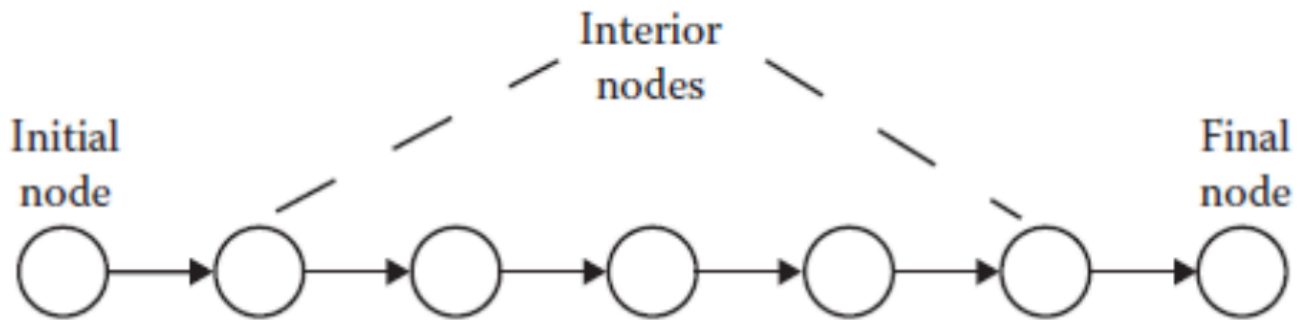


- Non-executable statements are not included
- Nodes 4 to 8 are a sequence, nodes 9 to 12 are an if-then-else construct, nodes 13 to 22 are nested if-then-else construct.
- Node 4 is a program source that signify single-entry (indeg = 0)
- Node 23 is a sink node that signify single-exit (outdeg = 0)
- No loops exist, so this is a directed acyclic graph

DD-Paths

- The best known form of structural testing is based on a construct known as a decision-to-decision path.
最广为人知的结构化测试形式基于一种称为“决策到决策路径”的结构。
- A DD-Path is a chain obtained from a program graph. It is a sub-path **of the Control Flow Graph**.
DD-Path是从程序图中获得的一条链，它是控制流图的一个子路径。
- It is the path from one decision node to another, or from a decision node to a terminal node
这是从一个决策节点到另一个决策节点，或从一个决策节点到终端节点的路径。
- More formally a DD-Path is a chain obtained from a program graph such that:
更正式地说，DD-路径（DD-Path）是从程序图中获得的链，其满足以下条件：

- Case1: it consists of a single node with $\text{indeg}=0$.
- Case2: it consists of a single node with $\text{outdeg}=0$,
- Case3: it consists of a single node with $\text{indeg} \geq 2$ or $\text{outdeg} \geq 2$ (for example, node D and A)
- Case4: it consists of a single node with $\text{indeg} = 1$, and $\text{outdeg} = 1$
- Case5: it is a maximal chain of length ≥ 1



- Given a program written in an imperative language, its DD-path graph is the directed graph in which nodes are DD-paths of its program graph, and edges represent control flow between successor DD-paths.

给定一个用命令式语言编写的程序，其DD路径图是一种有向图，其中节点对应程序图中的DD路径，而边则表示后继DD路径之间的控制流。

CFG to DD-Path Graph

CFG to DD-Path Graph

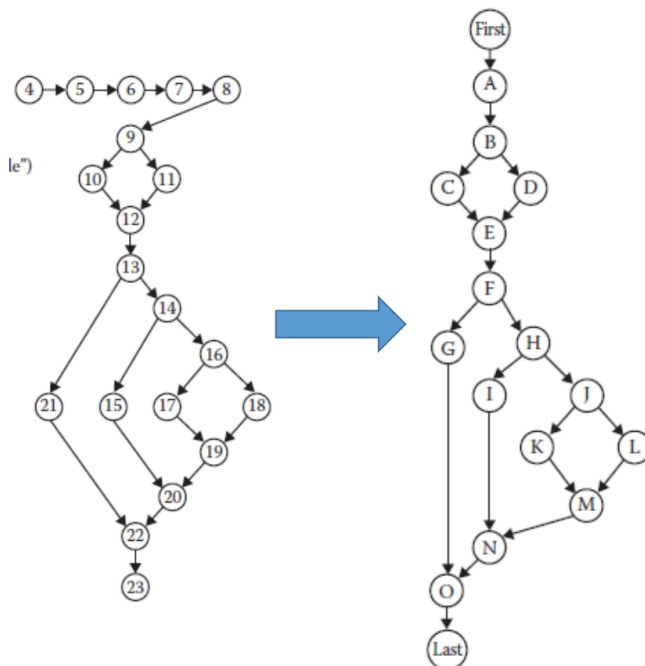


Figure 8.2
Nodes

DD-Path

Case of
definition

4	First	1
5-8	A	5
9	B	3
10	C	4
11	D	4
12	E	3
13	F	3
14	H	3
15	I	4
16	J	3
17	K	4
18	L	4
19	M	3
20	N	3
21	G	4
22	O	3
23	Last	2

Test Coverage Metrics

- The motivation of using DD-paths is that they enable **very precise descriptions of test coverage**.
使用DD-路径的动机在于它们能够实现对测试覆盖率非常精确的描述。
- The fundamental limitations of specification-based testing is that it is impossible to know either the extent of redundancy or the possibility of gaps corresponding to the way a set of functional test cases exercises a program
基于规格说明的测试的一个根本局限性在于，无法确定一组功能测试用例对程序进行测试时，其冗余程度如何，也无法判断是否存在与程序执行路径相对应的测试缺口。
- Test coverage metrics are a device to measure the extend to which a set of test cases covers a program.
测试覆盖率指标是一种用来衡量测试用例集对程序覆盖程度的工具。

E.F. Miller's Test Coverage Metrics

<i>Metric</i>	<i>Description of Coverage</i>
C_0	Every statement
C_1	Every DD-path
C_{1p}	Every predicate to each outcome
C_2	C_1 coverage + loop coverage
C_d	C_1 coverage + every dependent pair of DD-paths
C_{MCC}	Multiple condition coverage
C_{ik}	Every program path that contains up to k repetitions of a loop (usually $k = 2$)
C_{stat}	"Statistically significant" fraction of paths
C_∞	All possible execution paths

Statement and Predicate Coverage Testing

- Statement coverage based testing aims to devise test cases that collectively exercise all statements in a program.
基于语句覆盖的测试旨在设计测试用例，使其能够共同执行程序中的所有语句。
- Predicate coverage (or branch coverage, or decision coverage) based testing aims to devise test cases that evaluate each simple predicate of the program to True and False. Here the term simple predicate refers to either a single predicate or a compound Boolean expression that is considered as a single unit

that evaluates to True or False. This amounts to traversing every edge in the DD-Path graph. • For example in predicate coverage for the condition.

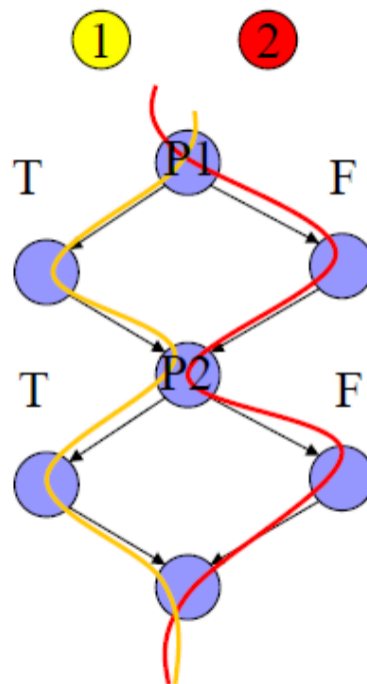
基于谓词覆盖（或分支覆盖、判定覆盖）的测试旨在设计测试用例，使程序中每个简单谓词取值True和False。此处的术语"简单谓词"指代可被视为单一评估单元并取值为True或False的单个谓词或复合布尔表达式。这等同于遍历DD-路径图中的每一条边。• 例如，在针对该条件的谓词覆盖测试中

- if(A or B) then C we could consider the test cases A=True, B= False (true case), and A=False, B=False (false case)

若满足条件A或B则执行C，可考虑测试用例：A=True, B=False（真值情况）及A=False, B=False（假值情况）。

DD-Path Graph Edge Coverage C_1

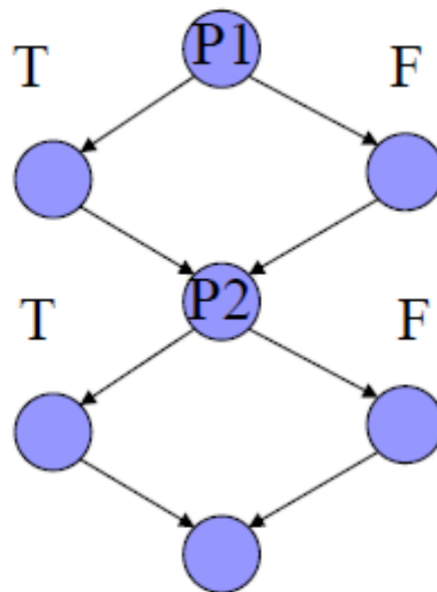
Here a T,T and F,F combination will suffice to have DD-Path Graph edge coverage or Predicate coverage C_1



DD-Path Coverage Testing C_{1p}

This is the same as the C_1 but now we must consider test cases that exercise **all possible outcomes** of the choices T,T, T,F, F,T, F,F for the predicates P1, and P2 respectively, in the DD-Path graph.

这与 C_1 相同，但现在我们必须考虑在DD路径图中，针对谓词P1和P2分别测试所有可能的选择结果T,T、T,F、F,T、F,F的测试用例。



Multiple Condition Coverage Testing 多条件覆盖测试

- Now if we consider that the predicates P1 is a compound predicate (i.e. (A or B)) then Multiple Condition Coverage Testing requires that **each possible combination of inputs be tested for each decision**.

现在考虑谓词P1是复合谓词（即A或B）的情况。对于这种由简单条件通过逻辑运算符连接而成的情况，多条件覆盖测试中的谓词“决定”指的是整个“决策”，而非单个条件。多条件覆盖测试要求对每个决策都测试所有可能的输入组合。

- Example: “if (A or B)” requires 4 test cases:
 - A = True, B = True
 - A = True, B = False
 - A = False, B = True
 - A = False, B = False
- The problem: **For n conditions, 2^n test cases** are needed, and this grows exponentially with n|问题：若存在 n 个条件，则需要 2^n 个测试用例，且该数量随 n 呈指数级增长。

Dependent DD-Path Pairs Coverage Testing C_d

- In simple C_1 coverage criterion we are interested simply to traverse all edges in the DD-Path graph.
简单的 C_1 覆盖准则中，我们的目标仅是遍历DD-路径图中的所有边。
- If we enhance this coverage criterion by ensuring that we traverse dependent pairs of DD-Paths also we may have the chance of revealing more errors that are based on data flow dependencies.
如果我们通过确保遍历DD路径的依赖对来增强这一覆盖准则，我们可能会有机会揭示更多基于数据流依赖关系的错误。
- More specifically, two DD-Paths are said to be dependent iff there is a define/reference relationship between these DD-Paths, in which a variable is defined (receives a value) in one DD-Path and is referenced in the other.
更具体地说，两条DD路径之间被称为存在依赖关系，当且仅当它们之间存在定义/引用关系，即某变量在其中一条DD路径中被定义（获得值），而在另一条DD路径中被引用。
- In C_d testing we are interested on covering all edges of the DD-Path graph and all dependent DD-Path

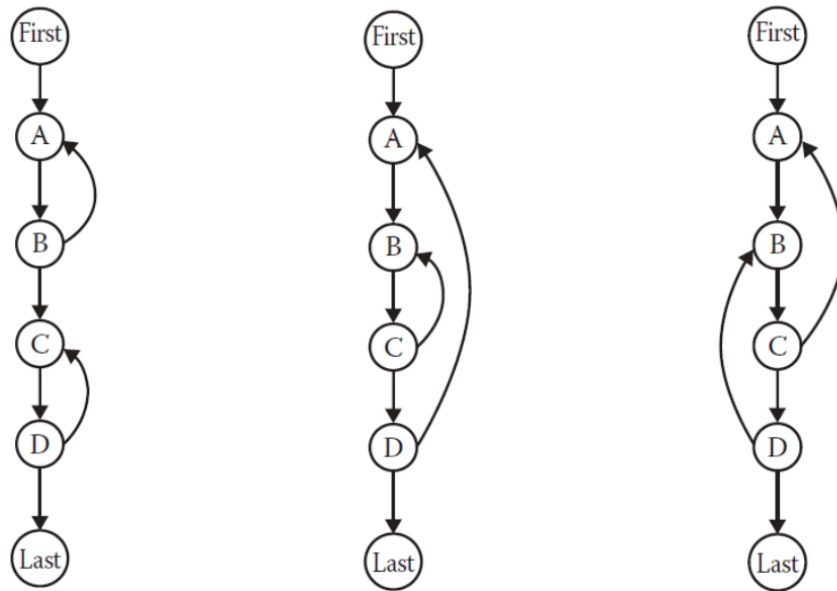
pairs.

在条件测试中，我们关注覆盖DD-路径图的所有边以及所有相互依赖的DD-路径对。

- The importance of these dependencies is that they are closely related to the problem of infeasible paths. We have good examples of dependent pairs of DD-paths: in slide 16, C and H are such a pair, as are DD-paths D and H.

这些依赖关系的重要性在于它们与不可达路径问题密切相关。我们已掌握DD-路径依赖对的典型示例：在第16张幻灯片中，C与H构成一对依赖关系，D与H同样如此。

Loop Coverage



Concatenated Loop, Nested Loop, and Knotted Loop

- The simple view of loop testing coverage is that we must devise test cases that exercise the two possible outcomes of the decision of a loop condition that is one to traverse the loop and the other to exit (or not enter) the loop.

循环测试覆盖的简单观点是，我们必须设计测试用例，以覆盖循环条件决策的两种可能结果：一种是遍历循环，另一种是退出（或不进入）循环。

- An extension would be to consider a modified boundary value analysis approach where the loop index is given a minimum, minimum +, a nominal, a maximum -, and a maximum value or even robustness testing.

一种扩展思路可以考虑采用修正的边界值分析方法，即对循环索引取最小值、最小值加、典型值、最大值减和最大值，甚至进行健壮性测试。

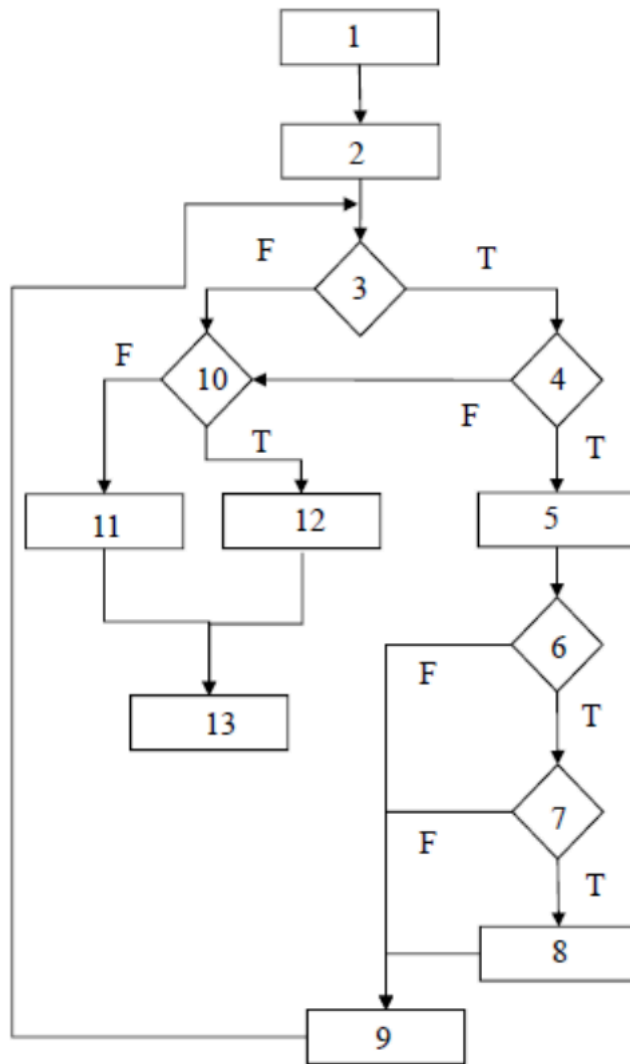
- In the case of nested loops we start with the inner most loop and we proceed outwards.

对于嵌套循环的情况，我们从最内层循环开始，然后逐步向外处理。

- If loops are knotted then we must apply data flow analysis testing techniques, that we will examine later in the module.

循环若存在嵌套，则须应用数据流分析测试技术，该技术将在本模块后续章节中详述。

Example



- Statement Coverage C0 :
 - SCPath1: 1-2-3(F)-10(F)-11-13
 - SCPath2: 1-2-3(T)-4(T)-5-6(T)-7(T)-8-9-3(F)-10(T)-12-13
- Branch or Decision Coverage C1 :
 - BCPath1: 1-2-3(F)-10(F)-11-13
 - BCPath2: 1-2-3(T)-4(T)-5-6(T)-7(T)-8-9-3(F)-10(T)-12-13
 - BCPath3: 1-2-3(T)-4(F)-10(F)-11-13
 - BCPath4: 1-2-3(T)-4(T)-5-6(F)-9-3(F)-10(F)-11-13
 - BCPath5: 1-2-3(T)-4(T)-5-6(T)-7(F)-9-3(F)-10(F)-11-13

Basic Path Testing

McCabe's Basis Path Testing

- Basis Path Testing is a white box testing approach proposed by Tom McCabe. The idea is to derive a measure of the logical complexity of a program and use this as a guide for defining a basic set of execution paths to test.

基本路径测试是汤姆·麦凯布提出的一种白盒测试方法。其核心理念在于计算程序逻辑复杂性的度量指标，并以此为依据确定一组需要测试的基础执行路径集合。

- The measure of complexity used is typically the cyclomatic complexity, which is based on the control flow structure of the program.

所使用的复杂度度量通常是圈复杂度，其基于程序的控制流结构。

- The method to carry out basis path testing has four steps:

基本路径测试方法的执行包含四个步骤：

1. Compute the program graph
计算程序图
2. Calculate the cyclomatic complexity
计算圈复杂度
3. Select a basis set of paths
选择一组路径基集
4. Generate test cases for each of these paths.
为每一条路径生成测试用例。

Step 1:

- To begin, we need a program graph from which to construct a basis
步骤1：首先，我们需要一个程序图来构建基。
- The below figure is a directed graph which is the DD-Path graph of some program.
下图是一个有向图，它是某个程序的DD路径图。
- The program has a single entry (A) and a single exit (G).
程序具有单个入口(A)和单个出口(G)。

Step 2:

- The cyclomatic complexity of a program, according to McCabe's method, is calculated as the number of linearly independent paths through the program's source code.
根据McCabe方法，程序的圈复杂度通过计算程序中线性独立路径的数量得出。
- Essentially, it provides a **count of the number of decisions** in the source code to give an indication of the program's overall complexity.
本质上，它通过对源代码中的决策数量进行计数，来反映出程序的整体复杂度。

-
- The formula for cyclomatic complexity is given by

圈复杂度的公式如下：

- $V(G) = e - n + 2p$ or $V(G) = e - n + p$
- Where, e: the number edges; n: the number of nodes; p: number of connected regions.
- 其中, e 为边数; n 为节点数; p 为连通区域数。

- The number of linearly independent paths from source node to sink node in the graph is

图中从源节点到汇节点的线性无关路径的数量为

- $V(G) = e - n + 2p = 10 - 7 + 2(1) = 5$

- The number of linearly independent circuits in the graph is

图中线形独立回路的数目为

- $V(G) = e - n + p = 11 - 7 + 1 = 5$

Step 3:

- A linearly independent path is any path through the source code that introduces at least one new set of processing statements or a new condition.

线性独立路径是指贯穿源代码的任意路径，该路径至少引入一组新的处理语句或一个新条件。

- McCabe's Baseline Method

- The method begins with the selection of a "baseline path", which should correspond to a normal execution of a program (from start node to end node, and has as many decisions as possible).

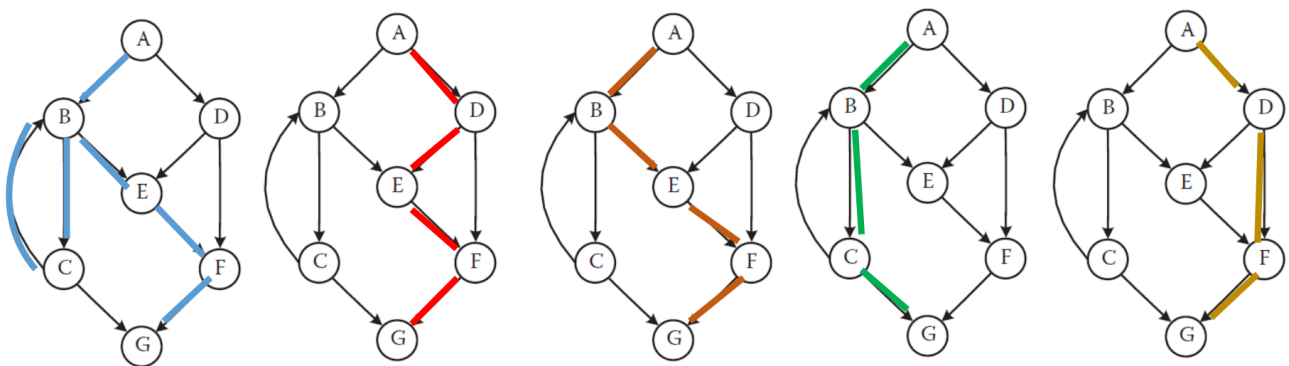
该方法首先选择一条“基线路径”，该路径应对应于程序的正常执行过程（从起始节点到终止节点，并包含尽可能多的决策分支）。

- The algorithm proceeds by retracing the paths visited and flipping the conditions one at a time.

该算法通过回溯已访问的路径并逐一翻转条件来实现。

- The process repeats up to the point all flips have been considered.

该过程持续重复，直至所有翻转情况均被考虑完毕。



Observation on Basis Path Method

- Two major soft spots occur in McCabe's view:

在麦凯布看来，存在两大明显薄弱环节：

- Testing the set of basis paths is sufficient (it is not!)

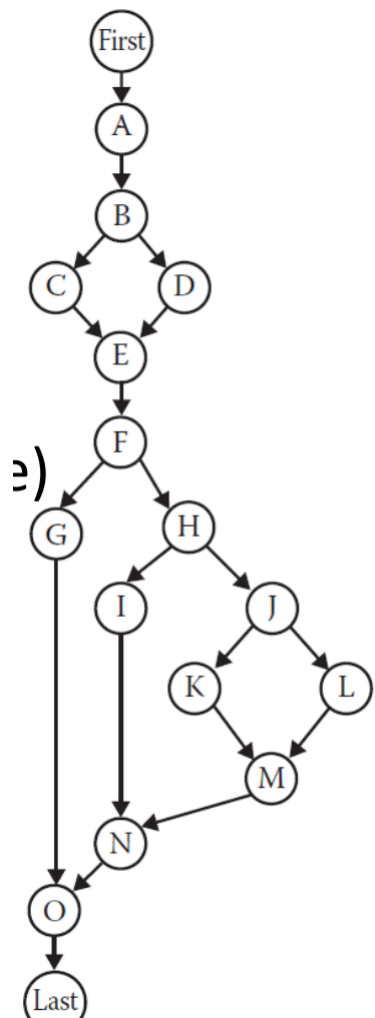
测试基本路径集是否充分（实际上并不充分！）

- Have to make program paths look like a vector space.

必须使程序路径看起来像向量空间。

- $V(G) = e - n + 2p = 18 - 15 + 2(1) = 5$

- Original p1: A-B-C-E-F-H-J-K-M-N-O-Last (Scalene)
- Flip p1 at B - p2: A-B-D-E-F-H-J-K-M-N-O-Last (Infeasible)
- Flip p1 at F - p3: A-B-C-E-F-G-O-Last (Infeasible)
- Flip p1 at H - p4: A-B-C-E-F-H-I-N-O-Last (Equilateral)
- Flip p1 at J - p5: A-B-C-E-F-H-J-L-M-N-O-Last (Isosceles)



- McCabe's procedure successfully identifies basis path that are topologically independent, but when these contradict semantic dependences, topologically possible paths are seen to be logically infeasible.
McCabe的过程成功识别出拓扑上独立的基路径，但当这些路径与语义依赖相矛盾时，拓扑上可能的路径会被视为逻辑上不可行的。
- If we assume that the logic of the program dictates that "if node C is traversed, then we must traverse node H, and if node D is traversed, then we must traverse Node G"
假设程序逻辑规定“若遍历节点C，则必须遍历节点H；若遍历节点D，则必须遍历节点G”。
- These constraints will eliminate Paths 2,3
这些限制条件将排除路径2和路径3。
 - We also need a basis path for the NotATriangle case

另外，我们还需要为NotATriangle情况设置一个基础路径。

- We are left to consider four feasible paths:

我们剩下需要考虑四种可行的路径：

- p1: A-B-C-E-F-H-J-K-M-N-O-Last (Scalene)
- p4: A-B-C-E-F-H-I-N-O-Last (Equilateral)
- p5: A-B-C-E-F-H-J-L-M-N-O-Last (Isosceles)
- p6: A-B-D-E-F-G-O-Last (Not a triangle)

Guidelines and Observations

- Functional testing focuses on properties that are “too far” or disassociated from the source code being tested

功能测试关注的是与待测源代码“距离过远”或已脱钩的属性。

- Path testing focuses too much on the graph and not the logic of the code, but is very useful to measure the quality of the testing (coverage criteria)

路径测试过度关注控制流图而非代码逻辑，但在评估测试质量（覆盖准则）方面具有重要实用价值。

- Basis path testing is giving us a lower bound of how much testing is necessary

基路径测试为我们提供了所需测试量的下界。

- Path testing is providing a set of metrics that act as cross checks to functional testing

路径测试提供了一组指标，作为对功能测试的一种交叉校验手段。

- When a program path is traversed by several functional test cases we suspect redundancy

当某程序路径被多个功能测试用例覆盖时，我们疑其存在冗余。

- When we fail to attain DD-Path graph coverage, we know that there are gaps in the functional test cases

当我们未能达到DD路径图覆盖率时，意味着功能测试用例存在遗漏。

- Coverage metrics can be used as

覆盖度指标可用作

- Setting minimum testing standards and as mechanism to selectively test portions of the code more rigorously than others

设置最低测试标准，并将其作为对不同代码部分进行差异化严格测试的机制。